



Cơ sở dữ liệu nâng cao



CHƯƠNG III

LÀM VIỆC VỚI KIỂU DỮ LIỆU VÀ MÃ KỊCH BẢN

Nội dung trình bày

1. **Làm việc với các kiểu dữ liệu**
2. **Mã kịch bản**



1. Làm việc với các kiểu dữ liệu

1.1 Chuyển đổi dữ liệu

- Khi làm việc với các biểu thức chứa nhiều kiểu dữ liệu khác nhau, phải thực hiện chuyển đổi giữa các kiểu dữ liệu.
- Hai loại chuyển đổi dữ liệu
 - Chuyển đổi ngầm (do SQL server tự thực hiện)
 - Chuyển đổi tường minh (sử dụng các hàm thư viện)

1.1.1 Chuyển đổi ngầm

- Các trường hợp xảy ra chuyển đổi ngầm
 - Gán giá trị cho một cột có kiểu dữ liệu khác với giá trị được gán.
 - Biểu thức tính toán có sự tham gia của nhiều loại dữ liệu khác nhau. Trong trường hợp này:
 - SQL Server chuyển đổi giá trị kiểu dữ liệu có độ ưu tiên thấp hơn sang kiểu dữ liệu có độ ưu tiên cao hơn.
- Lập trình viên cần hiểu về chuyển đổi ngầm dữ liệu để tránh thu được kết quả không mong muốn khi viết câu lệnh SQL

1.1.1 Chuyển đổi ngầm

- Ví dụ
 - Biểu thức tính toán có sự tham gia của nhiều kiểu dữ liệu khác nhau

InvoiceTotal * .0775

-- InvoiceTotal (kiểu money) bị chuyển đổi ngầm sang kiểu decimal

PaymentTotal – 100

-- Giá trị số 100 bị chuyển đổi ngầm sang kiểu money

1.1.1 Chuyển đổi ngầm

- Ví dụ
 - Gán giá trị có kiểu dữ liệu khác với kiểu dữ liệu của cột

PaymentDate = '2008-08-05'

-- Giá trị ngày có định dạng chuỗi '2008-08-05' bị chuyển đổi ngầm sang giá trị smalldatetime

1.1.1 Chuyển đổi ngầm

- Thứ tự ưu tiên của các kiểu dữ liệu phổ biến trong SQL Server

Độ ưu tiên	Nhóm	Kiểu dữ liệu
Cao nhất	Ngày/Giờ	datetime smalldatetime
	Số	float real decimal money smallmoney int smallint tinyint bit
Thấp nhất	Chuỗi	nvarchar nchar varchar char

1.1.1 Chuyển đổi ngầm

- Không phải tất cả kiểu dữ liệu đều có thể thực hiện chuyển đổi ngầm sang toàn bộ kiểu dữ liệu khác.

1.1.2 Chuyển đổi tường minh

- Khi thực hiện các chuyển đổi:
 - Không thể hoàn thành với chuyển đổi ngầm định
 - Chuyển dữ liệu có kiểu dữ liệu với độ ưu tiên cao hơn về kiểu dữ liệu có độ ưu tiên thấp hơn

ta sử dụng hàm CAST hoặc CONVERT để thực hiện phép chuyển đổi tường minh.

1.1.2.1 Hàm CAST

- Cú pháp hàm CAST

CAST(<Biểu thức> AS <Kiểu dữ liệu>[(length)])

- Trường hợp sử dụng:
 - Hàm **CAST** dùng để chuyển đổi tường minh, hay *ép kiểu*, một *biểu thức* từ kiểu dữ liệu này sang kiểu dữ liệu khác.

1.1.2.1 Hàm CAST

- Ví dụ
 - Câu lệnh SELECT sử dụng hàm CAST

```
SELECT InvoiceDate, InvoiceTotal,  
       CAST(InvoiceDate AS varchar) AS varcharDate,  
       CAST(InvoiceTotal AS int)     AS integerTotal,  
       CAST(InvoiceTotal AS varchar) AS varcharTotal  
FROM Invoices
```

	InvoiceDate	InvoiceTotal	varcharDate	integerTotal	varcharTotal
1	2008-04-08 00:00:00	3813.33	Apr 8 2008 12:00AM	3813	3813.33
2	2008-04-10 00:00:00	40.20	Apr 10 2008 12:00AM	40	40.20
3	2008-04-13 00:00:00	138.75	Apr 13 2008 12:00AM	139	138.75
4	2008-04-16 00:00:00	144.70	Apr 16 2008 12:00AM	145	144.70

1.1.2.1 Hàm CAST

- Ví dụ
 - Chuyển đổi dữ liệu khi thực hiện phép chia số nguyên

Thao tác	Kết quả
50/100	0 (Kiểu int)
50/CAST(100 AS decimal(3))	.500000 (Kiểu decimal)

1.1.2.2 Hàm Convert

- Cú pháp hàm CONVERT

CONVERT(<Kiểu dữ liệu>[(length)], <Biểu thức>[, <Tham số định dạng>])

- Trường hợp sử dụng
 - Sử dụng hàm **CONVERT** để chuyển đổi tương minh một biểu thức từ kiểu dữ liệu này sang kiểu dữ liệu khác
 - Khi chuyển đổi sang kiểu dữ liệu kí tự và cần định dạng hiển thị cho dữ liệu
- **Tham số định dạng**: định dạng hiển thị cho các giá trị ngày/giờ, số thực, tiền tệ khi chuyển đổi sang kiểu kí tự.
- Chỉ có **SQL Server** hỗ trợ hàm **CONVERT**. Hàm này ít được sử dụng hơn hàm **CAST** (Hàm chuẩn ANSI).

1.1.2.2 Hàm Convert

- Ví dụ
 - Câu lệnh **SELECT** sử dụng **CONVERT**

```
SELECT  CONVERT(varchar, InvoiceDate)           AS varcharDate,  
        CONVERT(varchar, InvoiceDate, 1)       AS varcharDate_1,  
        CONVERT(varchar, InvoiceDate, 107)     AS varcharDate_107,  
        CONVERT(varchar, InvoiceTotal)         AS varcharTotal,  
        CONVERT(varchar, InvoiceTotal, 1)      AS varcharTotal_1  
FROM Invoices
```

	varcharDate	varcharDate_1	varcharDate_107	varcharTotal	varcharTotal_1
1	Apr 8 2008 12:00AM	04/08/08	Apr 08, 2008	3813.33	3,813.33
2	Apr 10 2008 12:00AM	04/10/08	Apr 10, 2008	40.20	40.20
3	Apr 13 2008 12:00AM	04/13/08	Apr 13, 2008	138.75	138.75
4	Apr 16 2008 12:00AM	04/16/08	Apr 16, 2008	144.70	144.70

1.1.2.2 Hàm Convert

- Sinh viên tham khảo các mã tham số định dạng cho câu lệnh **CONVERT** trên Website sau:
 - <https://docs.microsoft.com/en-us/sql/t-sql/functions/cast-and-convert-transact-sql?view=sql-server-ver15>

1.2 Làm việc với kiểu dữ liệu chuỗi

Hàm	Mô tả
LEN(string)	Trả về số lượng ký tự trong chuỗi, tính cả ký tự trắng đầu chuỗi nhưng không bao gồm ký tự trắng cuối chuỗi.
LTRIM(string)	Trả về chuỗi với các ký tự trắng đầu chuỗi bị loại bỏ.
RTRIM(string)	Trả về chuỗi với các ký tự trắng cuối chuỗi bị loại bỏ.
LEFT(string, length)	Trả về chuỗi con có chiều dài length tính từ đầu chuỗi
RIGHT(string, length)	Trả về chuỗi con có chiều dài length tính từ cuối chuỗi
SUBSTRING(string, start, length)	Trả về chuỗi con có chiều dài length tính từ vị trí start
CHARINDEX(find, search[, start])	Trả về vị trí xuất hiện đầu tiên của chuỗi find trong chuỗi search bắt đầu từ vị trí start. Trả về 0 nếu không tìm thấy.
PATINDEX(find, search)	Tương tự hàm CHARINDEX nhưng chuỗi mẫu find có thể bao gồm các ký tự thay thế.
REPLACE(search, find, replace)	Trả về chuỗi search với tất cả các chuỗi find được thay thế bởi chuỗi replace.
REVERSE(string)	Trả về chuỗi với các ký tự đảo ngược.
LOWER(string)	Trả về chuỗi được chuyển đổi thành các chữ cái thường.
UPPER(string)	Trả về chuỗi được chuyển đổi thành các chữ cái hoa.
SPACE(integer)	Trả về chuỗi với số lượng ký tự trắng được chỉ định qua tham số integer

1.2 Làm việc với kiểu dữ liệu chuỗi

- Ví dụ

Hàm	Kết quả
<code>LEN('SQL Server')</code>	10
<code>LEN(' SQL Server ')</code>	12
<code>LEFT('SQL Server', 3)</code>	'SQL'
<code>LTRIM(' SQL Server ')</code>	'SQL Server '
<code>RTRIM(' SQL Server ')</code>	' SQL Server'
<code>LTRIM(RTRIM(' SQL Server '))</code>	'SQL Server'
<code>LOWER('SQL Server')</code>	'sql server'
<code>UPPER('ca')</code>	CA
<code>PATINDEX('%v_r%', 'SQL Server')</code>	8
<code>CHARINDEX('SQL', ' SQL Server')</code>	3
<code>CHARINDEX('-', '(559) 555-1212')</code>	10
<code>SUBSTRING('(559) 555-1212', 7, 8)</code>	555-1212
<code>REPLACE(RIGHT('(559) 555-1212', 13), ') ', '-')</code>	559-555-1212

1.2 Làm việc với kiểu dữ liệu chuỗi

- Ví dụ
 - Câu lệnh **SELECT** sử dụng các hàm xử lý chuỗi **LEFT**, **RIGHT** và **SUBSTRING**

```
SELECT VendorName, VendorContactLName + ', ' +  
    LEFT(VendorContactFName, 1) + '!' AS ContactName,  
    RIGHT(VendorPhone, 8) AS Phone  
FROM Vendors  
WHERE SUBSTRING(VendorPhone, 2, 3) = 559  
ORDER BY VendorName
```

	VendorName	ContactName	Phone
1	Abbey Office Furnishings	Francis, K.	555-8300
2	BFI Industries	Kaleigh, E.	555-1551
3	Bill Marvin Electric Inc	Hostlery, K.	555-5106
4	Cal State Termite	Hunter, D.	555-1534
5	California Business Machines	Rohansen, A.	555-5570

1.3 Hướng dẫn giải quyết vấn đề thường xảy ra với dữ liệu chuỗi

- 1. Sắp thứ tự
 - Khi sắp xếp một cột kiểu chuỗi chứa số, bạn có thể nhận được kết quả không mong đợi.
 - Để tránh điều đó, bạn chuyển đổi cột kiểu chuỗi thành giá trị số trong mệnh đề ORDER BY.
- Hướng dẫn sử dụng hàm **CAST** để sắp xếp cột kiểu chuỗi chứa các số

- **Bảng StringSample được sắp xếp**

```
SELECT * FROM StringSample
```

```
ORDER BY ID
```

	ID	Name	AltID
1	1	Lizbeth Darien	01
2	17	Lance Pinos-Potter	17
3	2	Damell O'Sullivan	02
4	20	Jean Paul Renard	20
5	3	Alisha von Strump	03

- **Bảng StringSample được sắp xếp bởi cột ID được ép kiểu thành số nguyên**

```
SELECT * FROM StringSample
```

```
ORDER BY CAST(ID AS int)
```

	ID	Name	AltID
1	1	Lizbeth Darien	01
2	2	Damell O'Sullivan	02
3	3	Alisha von Strump	03
4	17	Lance Pinos-Potter	17
5	20	Jean Paul Renard	20

1.3 Hướng dẫn giải quyết vấn đề thường xảy ra với dữ liệu chuỗi

- 2. Phân tách ký tự
 - Nếu chuỗi gồm hai hay nhiều thành phần, bạn có thể phân tách chuỗi thành những thành phần độc lập.
 - Sử dụng hàm CHARINDEX để định vị những ký tự phân tách. Sau đó, dùng hàm LEFT, RIGHT, SUBSTRING và LEN để trích ra những thành phần độc lập.
- Hướng dẫn sử dụng hàm chuỗi ký tự để tách chuỗi

SELECT Name,

LEFT(Name, CHARINDEX(' ', Name) - 1) AS First,

RIGHT(Name, LEN(Name) - CHARINDEX(' ', Name)) AS Last

FROM StringSample

	Name	First	Last
1	Lizbeth Darien	Lizbeth	Darien
2	Damell O'Sullivan	Damell	O'Sullivan
3	Lance Pinos-Potter	Lance	Pinos-Potter
4	Jean Paul Renard	Jean	Paul Renard
5	Alisha von Strump	Alisha	von Strump

1.3 Các hàm xử lý dữ liệu ngày/giờ

Hàm	Mô tả
GETDATE ()	Trả về giá trị datetime cho ngày và giờ hiện tại.
GETUTCDATE ()	Trả về giá trị datetime cho ngày và giờ hiện tại theo UTC dựa trên đồng hồ hệ thống và múi giờ được cài đặt.
SYSDATETIME ()	Trả về giá trị datetime2(7) cho ngày và giờ hiện tại.
SYSUTCDATETIME ()	Trả về giá trị datetime2(7) cho ngày và giờ hiện tại theo UTC dựa trên đồng hồ hệ thống và múi giờ được cài đặt.
SYSDATETIMEOFFSET ()	Trả về giá trị datetimeoffset(7) cho ngày và giờ hiện tại theo UTC dựa trên đồng hồ hệ thống và múi giờ được cài đặt, với múi giờ <i>không</i> được điều chỉnh theo ngày thay đổi múi giờ.
DAY (date)	Trả về ngày trong tháng theo kiểu số nguyên.
MONTH (date)	Trả về tháng theo kiểu số nguyên.
YEAR (date)	Trả về năm với 4 chữ số, theo kiểu số nguyên.
DATENAME (datepart, date)	
DATEPART (datepart, date)	Trả về số nguyên tương ứng với phần datepart (year, month...) của date
DATEADD (datepart, number, date)	Trả về ngày là kết quả phép cộng số lượng đơn vị datepart được chỉ định vào date.
DATEDIFF (datepart, startdate, enddate)	Trả về số đơn vị datepart giữa ngày bắt đầu và kết thúc được chỉ định.
ISDATE (expression)	Trả về giá trị 1 (true) nếu biểu thức là giá trị ngày/giờ; ngược lại, trả về giá trị 0 (false).

1.3 Các hàm xử lý dữ liệu ngày/giờ

- Các hàm **DATENAME**, **DATEPART**, **DATEADD**, **DATEDIFF** phải được truyền vào tham số datepart
- Các giá trị tham số **datepart** và dạng viết tắt tương ứng

Tham số	Dạng viết tắt
year	yy, yyyy
quarter	qq, q
month	mm, m
dayofyear	dy, y
day	dd, d
week	wk, ww
weekday	Dw
hour	Hh
minute	mi, n
second	ss, s
millisecond	Ms
microsecond	Mcs
Tzoffset	tz

1.3 Các hàm xử lý dữ liệu ngày/giờ

- Ví dụ

Hàm	Kết quả
<code>GETDATE ()</code>	2008-08-06 14:10:13.813
<code>GETUTCDATE ()</code>	2008-08-06 21:10:13.813
<code>SYSDATETIME ()</code>	2008-08-06 14:10:13.8160822
<code>SYSUTCDATETIME ()</code>	2008-08-06 21:10:13.8160822
<code>SYSDATETIMEOFFSET ()</code>	2008-08-06 14:10:13.8160822-07.00
<code>MONTH ('2008-09-30')</code>	9
<code>DAY ('2008-09-30')</code>	30
<code>YEAR ('2008-09-30')</code>	2008
<code>ISDATE ('2008-09-30')</code>	1
<code>ISDATE ('2008-09-31')</code>	0
<code>ISDATE ('23:59:59')</code>	1
<code>ISDATE ('23:99:99')</code>	0

1.3 Các hàm xử lý dữ liệu ngày/giờ

- Ví dụ

Hàm	Kết quả
<code>GETDATE ()</code>	2008-08-06 14:10:13.813
<code>GETUTCDATE ()</code>	2008-08-06 21:10:13.813
<code>SYSDATETIME ()</code>	2008-08-06 14:10:13.8160822
<code>SYSUTCDATETIME ()</code>	2008-08-06 21:10:13.8160822
<code>SYSDATETIMEOFFSET ()</code>	2008-08-06 14:10:13.8160822-07.00
<code>MONTH ('2008-09-30')</code>	9
<code>DAY ('2008-09-30')</code>	30
<code>YEAR ('2008-09-30')</code>	2008
<code>ISDATE ('2008-09-30')</code>	1
<code>ISDATE ('2008-09-31')</code>	0
<code>ISDATE ('23:59:59')</code>	1
<code>ISDATE ('23:99:99')</code>	0

1.3 Các hàm xử lý dữ liệu ngày/giờ

- Ví dụ

Hàm	Kết quả
<code>DATEPART(day, '2008-09-30 11:35:00')</code>	30
<code>DATEPART(month, '2008-09-30 11:35:00')</code>	9
<code>DATEPART(year, '2008-09-30 11:35:00')</code>	2008
<code>DATEPART(hour, '2008-09-30 11:35:00')</code>	11
<code>DATEPART(minute, '2008-09-30 11:35:00')</code>	35
<code>DATEPART(second, '2008-09-30 11:35:00')</code>	0
<code>DATEPART(quarter, '2008-09-30 11:35:00')</code>	3
<code>DATEPART(dayofyear, '2008-09-30 11:35:00')</code>	273
<code>DATEPART(week, '2008-09-30 11:35:00')</code>	40
<code>DATEPART(weekday, '2008-09-30 11:35:00')</code>	3
<code>DATEPART(milliseconds, '11:35:00.1234567')</code>	123
<code>DATEPART(microsecond, '11:35:00.1234567')</code>	123456
<code>DATEPART(nanosecond, '11:35:00.1234567')</code>	123456700
<code>DATEPART(tzoffset, '11:35:00.1234567 -07:00')</code>	-420

1.3 Các hàm xử lý dữ liệu ngày/giờ

- Ví dụ

Hàm	Kết quả
<code>DATENAME(day, '2008-09-30 11:35:00')</code>	30
<code>DATENAME(month, '2008-09-30 11:35:00')</code>	September
<code>DATENAME(quarter, '2008-09-30 11:35:00')</code>	3
<code>DATENAME(weekday, '2008-09-30 11:35:00')</code>	Saturday
<code>DATEADD(day, 1, '2008-09-30 11:35:00')</code>	2008-10-01 11:35:00.000
<code>DATEADD(week, 1, '2008-09-30 11:35:00')</code>	2008-10-07 11:35:00.000
<code>DATEDIFF(day, '2007-12-01', '2008-09-30')</code>	303
<code>DATEDIFF(month, '2007-12-01', '2008-09-30')</code>	9
<code>DATEDIFF(minute, '06:46:45', '11:35:00')</code>	289
<code>DATEDIFF(second, '06:46:45', '11:35:00')</code>	17295
<code>DATEDIFF(quarter, '2007-12-01', '2008-09-30')</code>	3
<code>DATEDIFF(week, '2007-12-01', '2008-09-30')</code>	44
<code>DATEDIFF(day, '2008-09-30', '2007-12-01')</code>	-303

1.4 Hướng dẫn thực hiện tìm kiếm theo thành phần ngày

- Giá trị thời gian thường chứa cả thành phần ngày và giờ. Do đó việc tìm kiếm theo riêng thành phần ngày và giờ có thể gặp khó khăn.
- Với dữ liệu trong bảng **DateSample** như bên dưới.

	ID	StartDate
1	1	1979-03-01 00:00:00.000
2	2	1999-02-28 00:00:00.000
3	3	2003-10-31 00:00:00.000
4	4	2005-02-28 10:00:00.000
5	5	2008-02-28 13:58:32.823
6	6	2008-03-01 09:02:25.000

→ Cột StartDate có kiểu dữ liệu datetime

- Câu lệnh SELECT tìm kiếm theo thành phần ngày sau sẽ không trả về kết quả.

```
SELECT * FROM DateSample WHERE StartDate = '2008-02-28'
```

1.4 Hướng dẫn thực hiện tìm kiếm theo thành phần ngày

- Câu lệnh **SELECT** sử dụng hàm **CONVERT** loại bỏ thành phần giờ (SQL Server 2008 hoặc phiên bản mới hơn)

```
SELECT * FROM DateSample
```

```
WHERE CONVERT(date, StartDate) = '2008-02-28'
```

- Câu lệnh **SELECT** tìm kiếm theo miền của ngày

```
SELECT * FROM DateSample
```

```
WHERE StartDate >= '2008-02-28' AND StartDate < '2008-02-29'
```

- Câu lệnh **SELECT** sử dụng các hàm **MONTH**, **DAY**, and **YEAR** để tìm kiếm theo từng thành phần ngày, tháng, năm

- ```
SELECT * FROM DateSample
```

- ```
WHERE MONTH(StartDate) = 2 AND
```

- ```
DAY(StartDate) = 28 AND
```

- ```
YEAR(StartDate) = 2008
```

1.4 Hướng dẫn thực hiện tìm kiếm theo thành phần ngày

- Câu lệnh SELECT sử dụng hàm CAST để loại bỏ thành phần giờ

```
SELECT * FROM DateSample
```

```
WHERE CAST(CAST(StartDate AS char(11)) AS datetime) = '2008-02-28'
```

- Câu lệnh SELECT sử dụng hàm CONVERT để loại bỏ thành phần giờ

```
SELECT * FROM DateSample
```

```
WHERE CONVERT(datetime, CONVERT(char(10), StartDate, 110)) = '2008-02-28'
```

- Kỹ thuật thứ hai (tìm kiếm theo miền của ngày) là kỹ thuật duy nhất không sử dụng bất cứ hàm nào trong mệnh đề WHERE. Do đó, đây là kỹ thuật hiệu quả nhất trong việc tìm kiếm ngày.

1.5 Hướng dẫn thực hiện tìm kiếm theo thành phần giờ

- Với dữ liệu trong bảng DateSample như trước

- Hai điều kiện tìm kiếm sau không trả về kết quả

```
SELECT * FROM DateSample
```

```
WHERE StartDate = CAST('10:00:00' AS datetime)
```

```
SELECT * FROM DateSample
```

```
WHERE StartDate >= '09:00:00' AND StartDate < '12:59:59:999'
```

- Nguyên nhân:

- Giá trị giờ sẽ được chuyển đổi ngầm định sang kiểu date/time với thành phần ngày mặc định là 1900-01-01.

1.5 Hướng dẫn thực hiện tìm kiếm theo thành phần giờ

- Câu lệnh **SELECT** loại bỏ thành phần **ngày**. Sử dụng từ phiên bản SQL Server 2008.

```
SELECT * FROM DateSample
```

```
WHERE CONVERT(time, StartDate) >= '09:00:00' AND
```

```
CONVERT(time, StartDate) < '12:59:59:999'
```

	ID	StartDate
1	4	2005-02-28 10:00:00.000
2	6	2008-03-01 09:02:25.000

- Câu lệnh **SELECT** loại bỏ thành phần ngày. Sử dụng trong các phiên bản trước **SQL Server 2008**

```
SELECT * FROM DateSample
```

```
WHERE CONVERT(datetime, CONVERT(char(12), StartDate, 8)) >= '09:00:00'
```

```
AND CONVERT(datetime, CONVERT(char(12), StartDate, 8)) < '12:59:59:999'
```

	ID	StartDate
1	4	2005-02-28 10:00:00.000
2	6	2008-03-01 09:02:25.000

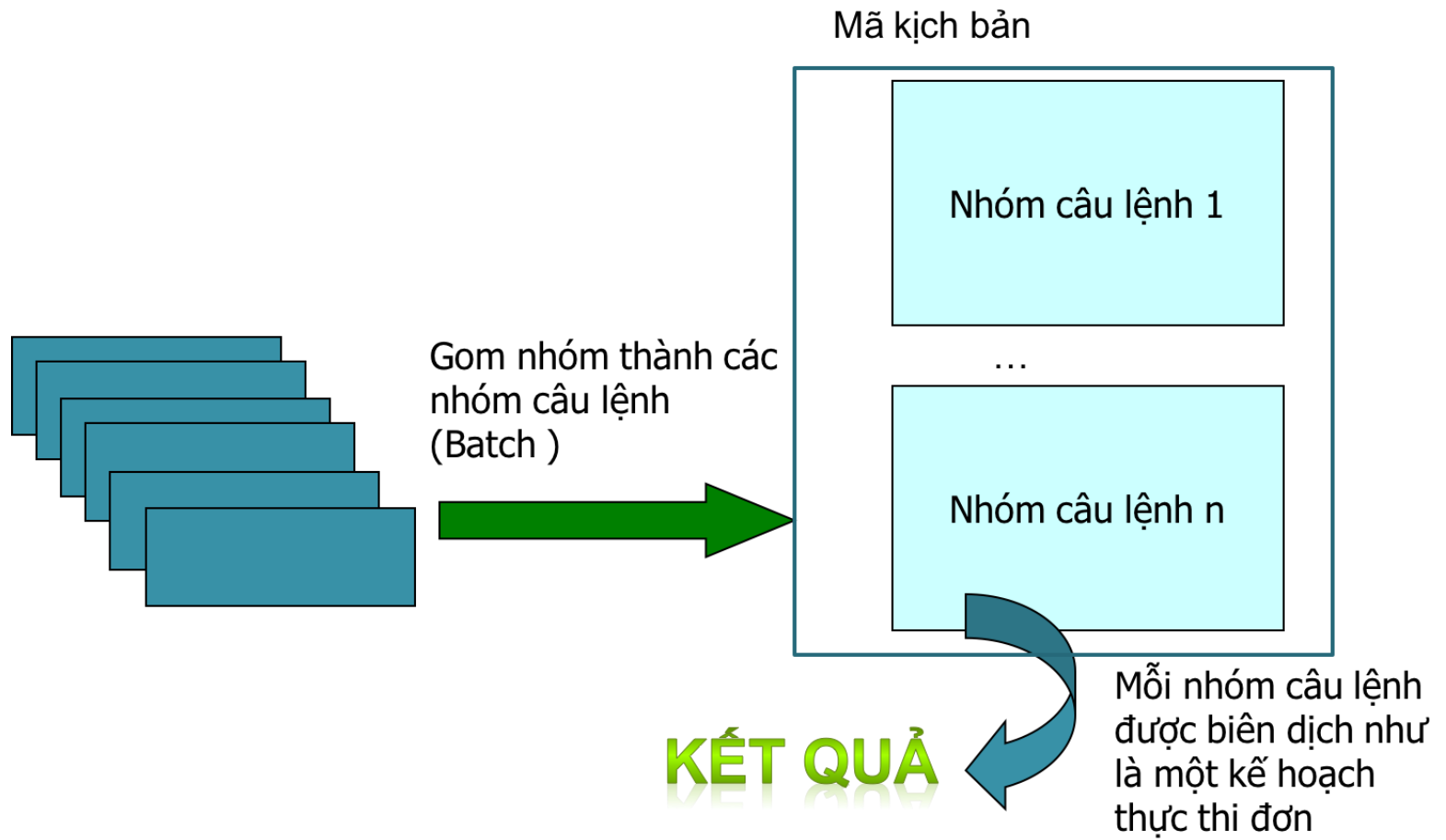
1.6 Chú ý khi lưu và truy vấn dữ liệu ngày/giờ

- Trong SQL Server dữ liệu ngày/giờ được xử lý dưới định dạng tháng/ngày/năm
- Để sử dụng định dạng ngày/giờ dưới dạng ngày/tháng/năm. Cần chú ý
 - Khi sử dụng câu lệnh INSERT phải truyền dữ liệu ngày/giờ theo định dạng tháng/ngày/năm
 - Khi truy vấn dữ liệu, để lấy về giá trị có định dạng ngày/tháng/năm có thể sử dụng hàm CONVERT với mã định dạng 3 hoặc 103



2. Mã kịch bản

2.1 Giới thiệu



2.1 Giới thiệu

- Lưu ý:
 - Tất cả các ví dụ bạn đã học ở bài hai đều là các mã kịch bản, gồm một nhóm câu lệnh

2.1 Giới thiệu

- Ví dụ

```
/*  
Tạo ba bảng trong cơ sở dữ liệu có tên ClubRoster.  
Tác giả:   Bryan Syverson  
Ngày tạo:  2006-08-12  
Ngày sửa:  2008-09-26  
*/
```

→ Chú thích cho
mã kịch bản.

```
CREATE DATABASE ClubRoster
```

→ Nhóm câu lệnh
(Batch) 1

```
GO
```

```
USE ClubRoster
```

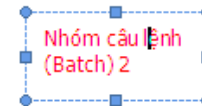
```
CREATE TABLE Members
```

```
(MemberID int NOT NULL IDENTITY PRIMARY KEY,
```

```
LastName varchar(75) NOT NULL,
```

```
FirstName varchar(50) NOT NULL,
```

```
MiddleName varchar(50) NULL)
```



```
CREATE TABLE Committees
```

```
(CommitteeID int NOT NULL IDENTITY PRIMARY KEY,
```

```
CommitteeName varchar(50) NOT NULL)
```

```
CREATE TABLE CommitteeAssignments
```

```
(MemberID int NOT NULL REFERENCES Members(MemberID),
```

```
CommitteeID int NOT NULL REFERENCES Committees(CommitteeID))
```

Lệnh Go đánh dấu
điểm kết thúc của
một nhóm câu
lệnh.

**Mã kịch bản tạo ba bảng “Members”, “Committees”, “CommitteeAssignment”
trong CSDL “ClubRoster”**

2.2 Các câu lệnh T-SQL sử dụng trong mã kịch bản

- Các câu lệnh T-SQL xử lý mã kịch bản

Từ khóa	Mô tả
USE	Thay đổi cơ sở dữ liệu hiện thời thành cơ sở dữ liệu được chỉ định trong câu lệnh
PRINT	Trả về thông báo tới client.
DECLARE	Định nghĩa biến cục bộ.
SET	Gán giá trị cho biến cục bộ hoặc biến theo phiên làm việc.
EXEC	Thực thi câu lệnh SQL hoặc stored procedure động.

Đã
học

2.2 Các câu lệnh T-SQL sử dụng trong mã kịch bản

- Các câu lệnh T-SQL điều khiển luồng thực thi

Từ khóa	Mô tả
IF...ELSE	Điều khiển luồng thực thi dựa vào điều kiện.
BEGIN...END	Định nghĩa khối câu lệnh.
WHILE	Lặp lại các câu lệnh chừng nào điều kiện được chỉ định là đúng.
BREAK	Thoát khỏi vòng lặp WHILE trong cùng.
CONTINUE	Quay về điểm bắt đầu vòng lặp WHILE .
TRY...CATCH	Điều khiển luồng thực thi khi có lỗi xảy ra.
GOTO	Thay đổi luồng thực thi vô điều kiện.
RETURN	Thoát vô điều kiện.

Đã
học

2.2 Các câu lệnh T-SQL sử dụng trong mã kịch bản

- Cú pháp của câu lệnh USE
USE <Tên CSDL>
- Cú pháp của câu lệnh PRINT
PRINT <Biểu thức chuỗi>

2.2 Các câu lệnh T-SQL sử dụng trong mã kịch bản

- Ví dụ về mã kịch bản sử dụng câu lệnh các câu lệnh T-SQL trên

USE AP

-- Khai báo biến @TotalDue

DECLARE @TotalDue money

-- Gán giá trị biến @TotalDue bằng tổng dư nợ chưa thanh toán của tất cả hóa đơn trong bảng

-- Invoices

```
SET @TotalDue = (SELECT SUM(InvoiceTotal -  
                        PaymentTotal - CreditTotal)  
                FROM Invoices)
```

-- Kiểm tra tổng dư nợ chưa thanh toán

-- Nếu lớn hơn 0 in ra dòng chữ "Total invoices due = \$" cùng với giá trị tổng dư nợ chưa thanh toán

-- được lưu trong biến @TotalDue

-- Nếu nhỏ hơn 0 in ra dòng chữ Invoices Paid in full

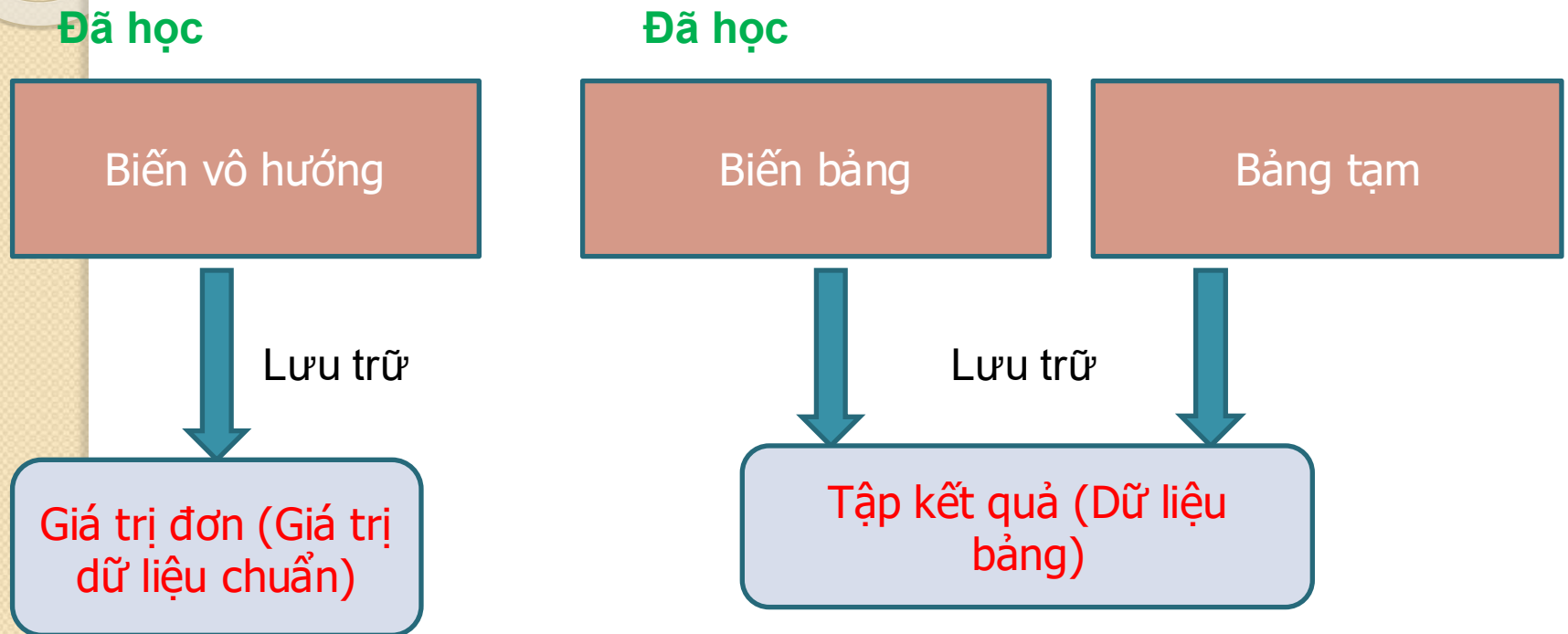
```
IF @TotalDue > 0
```

```
    PRINT 'Total invoices due = $' + CONVERT(varchar,@TotalDue,1)
```

```
ELSE
```

```
    PRINT 'Invoices paid in full'
```

2.3 Lưu trữ giá trị trong mã kịch bản



2.4 Bảng tạm

- Dùng để lưu trữ một tập kết quả trả về (dữ liệu dưới dạng bảng) từ một câu lệnh SELECT
- Hai loại bảng tạm:
 - Bảng tạm cục bộ
 - Tên bắt đầu bằng dấu #
 - Tồn tại trong phiên làm việc hiện tại. Trong phạm vi cửa sổ soạn thảo truy vấn tạo ra bảng đó
 - Bảng tạm toàn cục
 - Tên bắt đầu bằng dấu ##
 - Tồn tại trong tất cả các phiên làm việc.
- Bảng tạm được lưu trong CSDL tempdb
- Để xóa một bảng tạm, sử dụng lệnh DROP TABLE

2.4 Bảng tạm

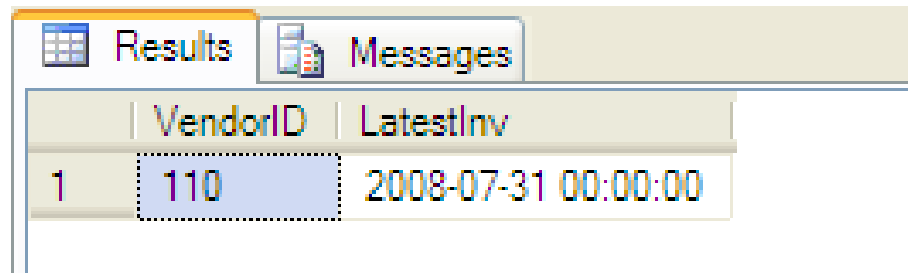
- Ví dụ sử dụng bảng tạm

-- Lưu tập kết quả tra về của câu lệnh SELECT vào bảng tạm cục bộ #TopVendors

```
SELECT TOP 1 VendorID, AVG(InvoiceTotal) AS AvgInvoice  
INTO #TopVendors  
FROM Invoice  
GROUP BY VendorID  
ORDER BY AvgInvoice DESC
```

-- Truy xuất dữ liệu được lưu trong bảng tạm

```
SELECT Invoices.VendorID, MAX(InvoiceDate) AS LatestInv  
FROM Invoices JOIN #TopVendors  
ON Invoices.VendorID = #TopVendors.VendorID  
GROUP BY Invoices.VendorID
```



Results		Messages
	VendorID	LatestInv
1	110	2008-07-31 00:00:00

2.5 Xử lý lỗi

- Cú pháp lệnh **TRY ... CATCH**

BEGIN TRY

{<câu lệnh SQL>|<Khởi câu lệnh>}

END TRY

BEGIN CATCH

{<Câu lệnh SQL>|<Khởi câu lệnh>}

END CATCH

2.5 Xử lý lỗi

- Hoạt động tương tự các hàm xử lý lỗi trong các ngôn ngữ lập trình
- **Các hàm có thể dùng trong khối CATCH**

Hàm	Mô tả
ERROR_NUMBER()	Trả về mã lỗi.
ERROR_MESSAGE()	Trả về thông điệp lỗi.
ERROR_SEVERITY()	Trả về mức nghiêm trọng của lỗi.
ERROR_STATE()	Trả về trạng thái của lỗi.

- Câu lệnh **TRY ... CATCH** chỉ bắt và xử lý được các lỗi có mức nghiêm trọng từ 10-20

2.5 Xử lý lỗi

- Ví dụ

```
USE AP  
GO
```

```
-- Bắt và xử lý lỗi chèn dữ liệu vào bảng Invoices
```

```
BEGIN TRY
```

```
    INSERT Invoices
```

```
    VALUES (799, 'ZXK-799', '2008-07-01', 299.95, 0, 0,  
            1, '2008-08-01', NULL)
```

```
-- Nếu lệnh chèn thực thi thành công in ra dòng bên dưới
```

```
    PRINT 'SUCCESS: Record was inserted.'
```

```
END TRY
```

```
-- Nếu có lỗi xảy ra khi chèn dữ liệu in ra dòng thông báo lỗi cùng với thông tin mã lỗi và thông báo lỗi
```

```
BEGIN CATCH
```

```
    PRINT 'FAILURE: Record was not inserted.'
```

```
    PRINT 'Error ' + CONVERT(varchar, ERROR_NUMBER(), 1)  
          + ':' + ERROR_MESSAGE()
```

```
END CATCH
```

Messages

FAILURE: Record was not inserted.

Error 245: Conversion failed when converting the varchar value 'ZXK-799' to data type int.

2.6 Sử dụng các hàm hệ thống

Tên hàm	Mô tả
@@IDENTITY	Trả về giá trị cuối cùng được tạo cho một cột định danh trên server. Trả về NULL nếu không có giá trị định danh nào được tạo.
IDENT_CURRENT('tablename')	Tương tự @@IDENTITY, nhưng trả về giá trị định danh cuối cùng được tạo cho một bảng xác định.
@@ROWCOUNT	Trả về số hàng chịu ảnh hưởng bởi câu lệnh SQL gần nhất.
@@ERROR	Trả về mã số lỗi được tạo bởi việc thực thi của câu lệnh SQL gần nhất. Trả về 0 nếu không xảy ra lỗi.
@@SERVERNAME	Trả về tên server cục bộ.
HOST_NAME()	Trả về tên trạm làm việc hiện thời.
SYSTEM_USER	Trả về tên người dùng hiện thời.

2.6 Sử dụng các hàm hệ thống

- Ví dụ

USE AP

DECLARE @MyIdentity int, @MyRowCount int

INSERT Vendors (VendorName, VendorAddress1, VendorCity, VendorState,
VendorZipCode, VendorPhone, DefaultTermsID,
DefaultAccountNo)

VALUES ('Peerless Binding', '1112 S Windsor St', 'Hallowell', 'ME',
'04347', '(207) 555-1555', 4, 400)

/* Gán giá trị cho biến @MyIdentity là giá trị định danh được tự động sinh cho cột VendorID của bảng Vendors và @MyRowCount số hàng bị ảnh hưởng bởi câu lệnh INSERT */

SET @MyIdentity = @@IDENTITY

SET @MyRowCount = @@ROWCOUNT

-- Kiểm tra xem câu lệnh INSERT ở trên có thực hiện thành công?

IF @MyRowCount = 1

INSERT Invoices

VALUES (200, @MyIdentity, 'BA-0199', '2008-08-01', 4598.23,
0, 0, 4, '2008-09-06', NULL, NULL)

2.6 Sử dụng các hàm hệ thống

- Ví dụ

USE AP

DECLARE @MyIdentity int, @MyRowCount int

INSERT Vendors (VendorName, VendorAddress1, VendorCity, VendorState,
VendorZipCode, VendorPhone, DefaultTermsID,
DefaultAccountNo)

VALUES ('Peerless Binding', '1112 S Windsor St', 'Hallowell', 'ME',
'04347', '(207) 555-1555', 4, 400)

/* Gán giá trị cho biến @MyIdentity là giá trị định danh được tự động sinh cho cột VendorID của bảng Vendors và @MyRowCount số hàng bị ảnh hưởng bởi câu lệnh INSERT */

SET @MyIdentity = @@IDENTITY

SET @MyRowCount = @@ROWCOUNT

-- Kiểm tra xem câu lệnh INSERT ở trên có thực hiện thành công?

IF @MyRowCount = 1

INSERT Invoices

VALUES (200, @MyIdentity, 'BA-0199', '2008-08-01', 4598.23,
0, 0, 4, '2008-09-06', NULL, NULL)

Tổng kết

- Các nội dung đã học trong bài:
 - Làm việc với các kiểu dữ liệu
 - Khi làm việc với các biểu thức chứa nhiều kiểu dữ liệu khác nhau, phải thực hiện chuyển đổi giữa các kiểu dữ liệu.
 - Hai loại chuyển đổi dữ liệu
 - Chuyển đổi ngầm (do SQL server tự thực hiện)
 - Chuyển đổi tường minh (sử dụng các hàm thư viện)
 - Sử dụng hàm CAST hoặc CONVERT
 - Hai trường hợp chuyển đổi ngầm
 - Gán giá trị cho một cột có kiểu dữ liệu khác với giá trị được gán.
 - Biểu thức tính toán có sự tham gia của nhiều loại dữ liệu khác nhau
 - Chuyển đổi ngầm từ kiểu dữ liệu có độ ưu tiên thấp hơn sang kiểu dữ liệu có độ ưu tiên cao hơn

Tổng kết

- SQL Server cung cấp các hàm làm việc với kiểu dữ liệu chuỗi, số, ngày giờ
- Các vấn đề xảy ra khi làm việc với kiểu dữ liệu chuỗi
 - Sắp thứ tự cho kiểu dữ liệu chuỗi chứa dữ liệu số
 - Sử dụng hàm CAST để chuyển đổi sang kiểu số trong mệnh đề ORDER BY
 - Phân tách dữ liệu chuỗi lưu trong một cột thành nhiều thành phần
 - Sử dụng các hàm CHARINDEX, LEFT, RIGHT, SUBSTRING, LEN
- Các vấn đề xảy ra với kiểu dữ liệu ngày giờ
 - Do kiểu dữ liệu ngày giờ thường chứa cả ngày và giờ -> Khi tìm kiếm theo thành phần ngày hoặc giờ, dễ không trả về kết quả
 - Sử dụng các hàm chuyển đổi dữ liệu, hàm trích xuất thành phần ngày, giờ hoặc tìm kiếm theo phạm vi

Tổng kết

- Mã kịch bản
 - Tập hợp các câu lệnh riêng lẻ được gom nhóm thành nhóm câu lệnh.
 - Mỗi nhóm câu lệnh được đánh dấu kết thúc bởi lệnh GO
 - Một mã kịch bản chứa từ một đến nhiều nhóm câu lệnh
 - Các câu lệnh T-SQL sử dụng trong mã kịch bản
 - Các câu lệnh xử lý mã kịch bản: USE, PRINT, DECLARE...
 - Các câu lệnh điều khiển luồng thực thi: IF...ELSE, TRY...CATCH, WHILE...
 - Lưu trữ giá trị trong mã kịch bản
 - Biến vô hướng
 - Biến bảng
 - Bảng tạm

HẾT CHƯƠNG III

Thank you for listening!